# Applied AI

Introduction to the Transformer

# Learning Objectives

By the end of the lesson today, you will be able to:

- Understand transformer architecture.
- Compare model families with real Hugging Face checkpoints.
- Deconstruct `pipeline()` into manual steps.
- Build a beginner-friendly Keras transformer block.

# Recap

Last time we saw how to transform raw text into model-ready columns. This process of text-vectorisation looks like:

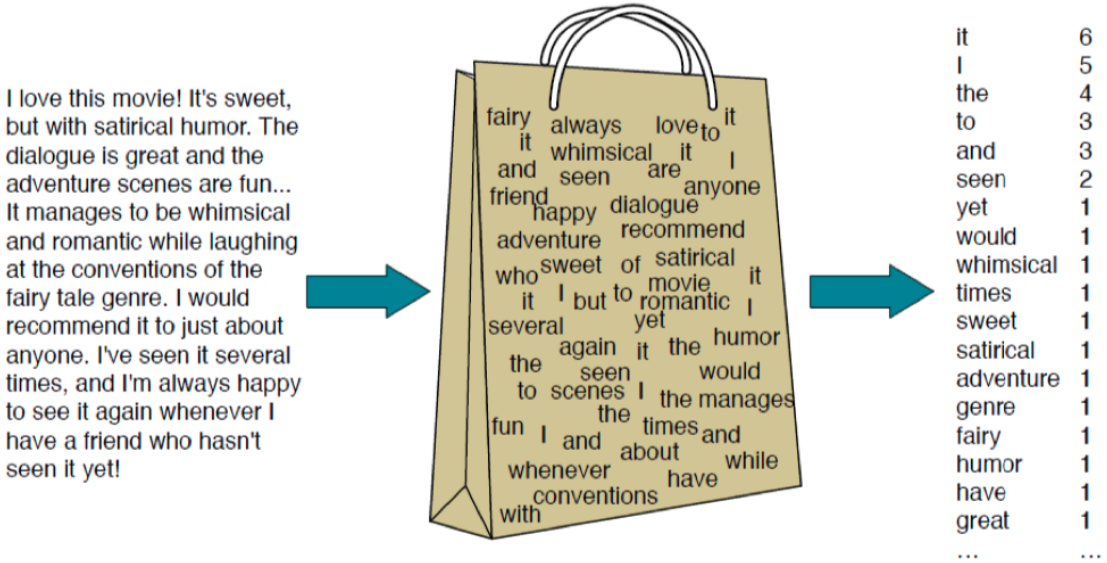
This week we will be looking at the model families and their forward behaviour. The process looks like:

Same mindset: inspect, verify, and document assumptions.

# Historic NLP Models

We spoke previously about how text vectorisatoin works slightly differently now to the past.

- Previously, more standisation was required.
- There was more focus on word-level tokenization.
- Models often relied on frequency of words to understand context. (Bag of words models)
- Models were often statistical in nature.




This Photo by Unknown Author is licensed under CC BY

# Pre-transformers: RNNs

As deep learning models became more powerful, the approach to NLP changed.

- Bag-of-words models were replaced with RNNs (recurrent neural networks).

These models were sequential in nature and had many different applications.

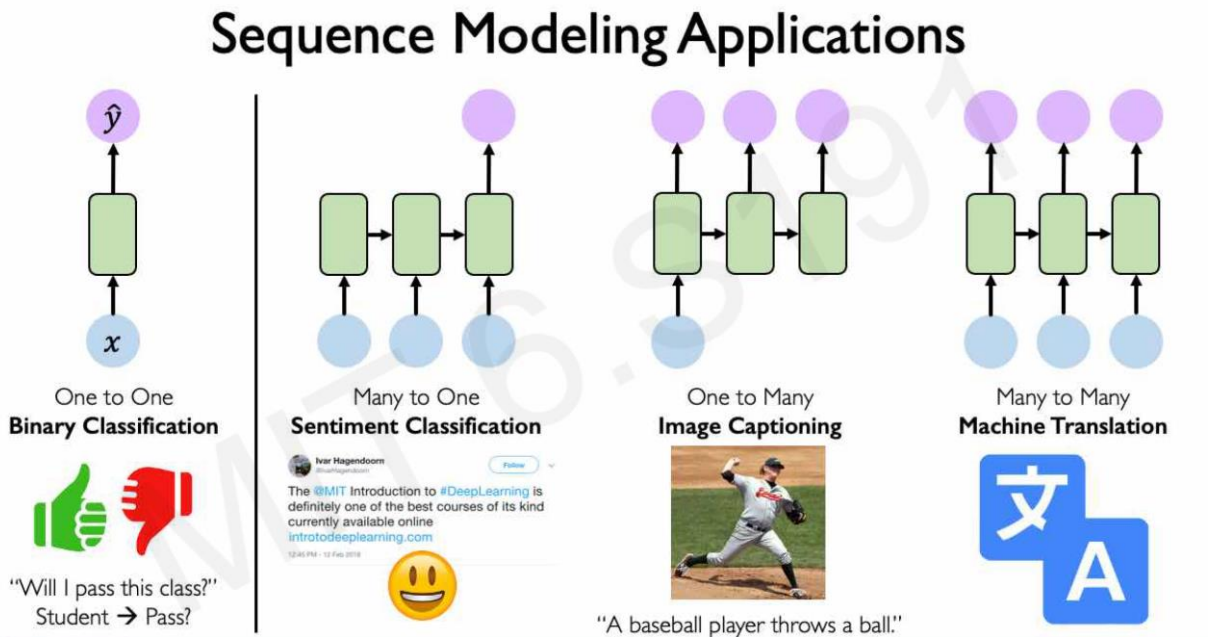



Image from: Deep Sequence Modelling, Lecture 2 Slides by E. Soleimany (MIT)

# Pre-transforms: RNNs

A recurrent neural network (RNN) is a type of neural network that is designed to process sequential data, such as time series or natural language.

RNNs are called "recurrent" because they have a feedback mechanism that allows them to "remember" previous inputs and use that information to make predictions about future inputs.

The basic component of an RNN is a recurrent unit which looks like the following:

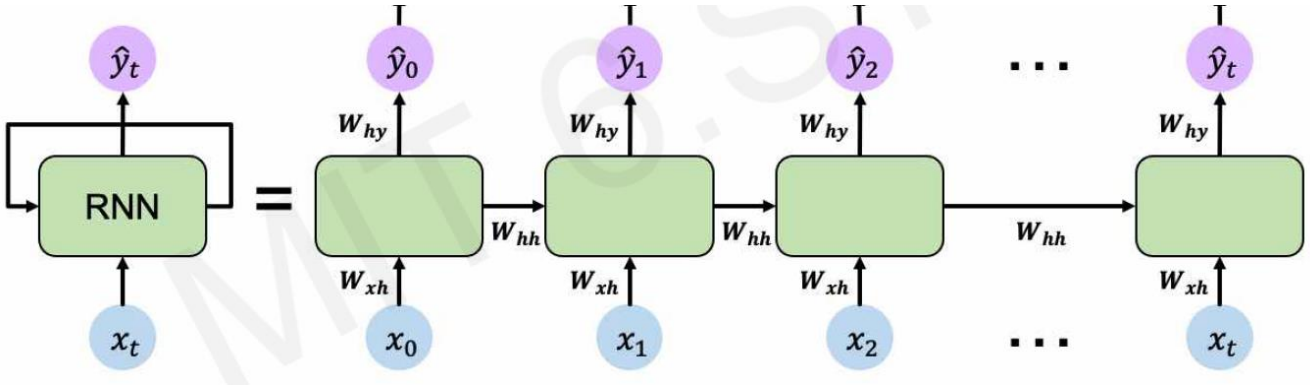



Image from: Deep Sequence Modelling, Lecture 2 Slides by E. Soleimany (MIT)

# Pre-transformers: RNNs

The RNN is built up from these simple recurrent units with one being the input to the recurrent unit.

- The processing is sequential.

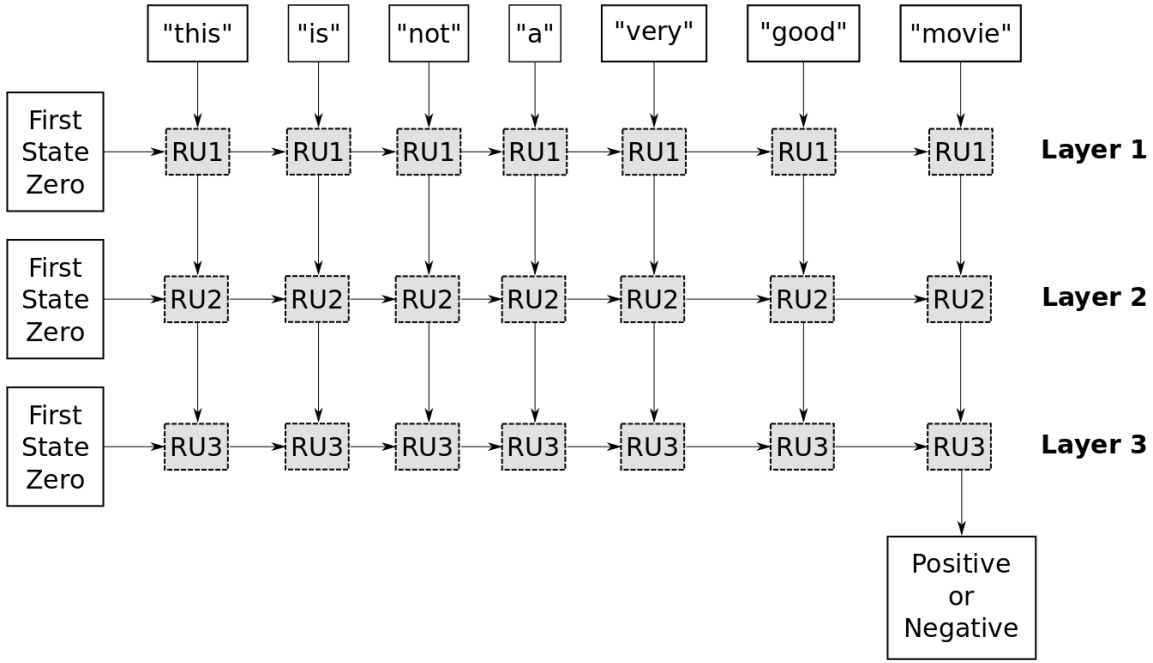



Image from: Natural Language Tutorial by M.E.H. Pedersen

# Pre-transformers: RNNs

Traditional Seq2Seq RNNs for tasks such as translation are encoder-decoder models.

- The entire input sentence is compressed into a single, fixed-size vector.
- This is then passed to the decoder RNN.

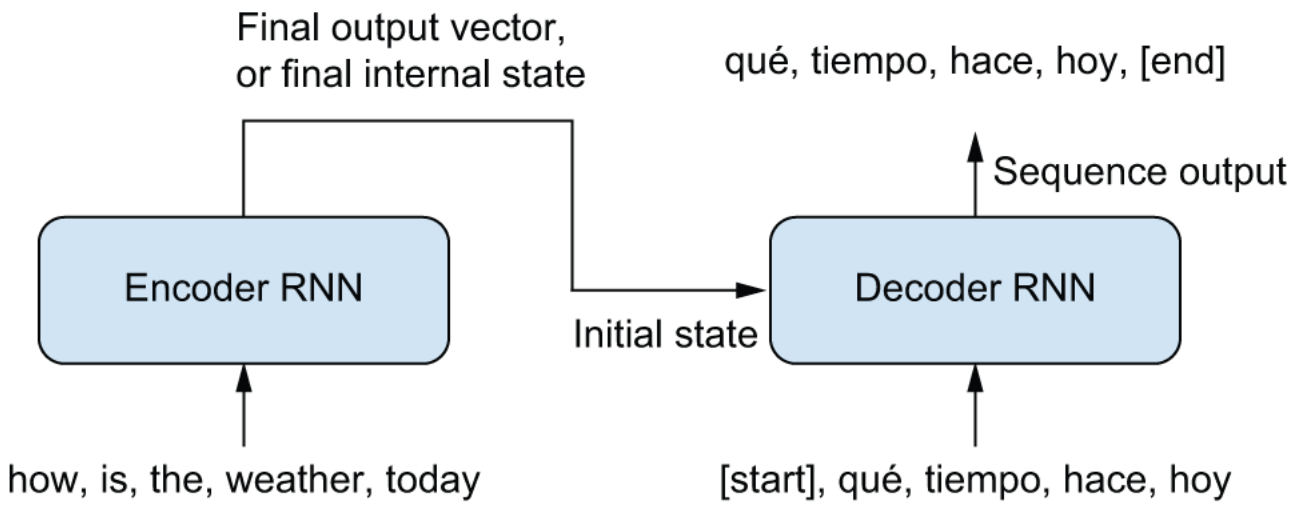



Image from: Chapter 15 of Deep Learning with Python by F. Chollet

# Why Transformers Replaced Sequential Bottlenecks

The problems encountered by these models are:

Sequential processing limits parallelism.

Long-range dependencies degrade memory of early tokens. (all info passes through hidden state and updated each time)

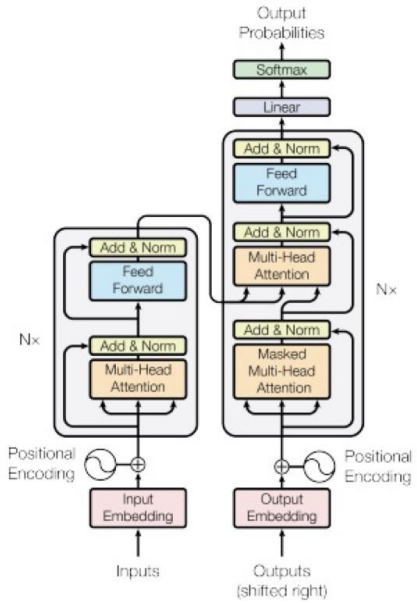
Particularly, in translation type problems issues with input being compressed to on fixed vector for decoder.

The transformer model overcomes these issues and we will go into this now.

**Attention Is All You Need**


Ashish Vaswani* Google Brain avaswani@google.com
Noam Shazeer* Google Brain noam@google.com
Niki Parmar* Google Research nikip@google.com
Jakob Uszkoreit* Google Research usz@google.com
Llion Jones* Google Research llion@google.com
Aidan N. Gomez* † University of Toronto aidan@cs.toronto.edu
Łukasz Kaiser* Google Brain lukaszkaiser@google.com
Illia Polosukhin* ‡ illia.polosukhin@gmail.com


https://arxiv.org/abs/1706.03762



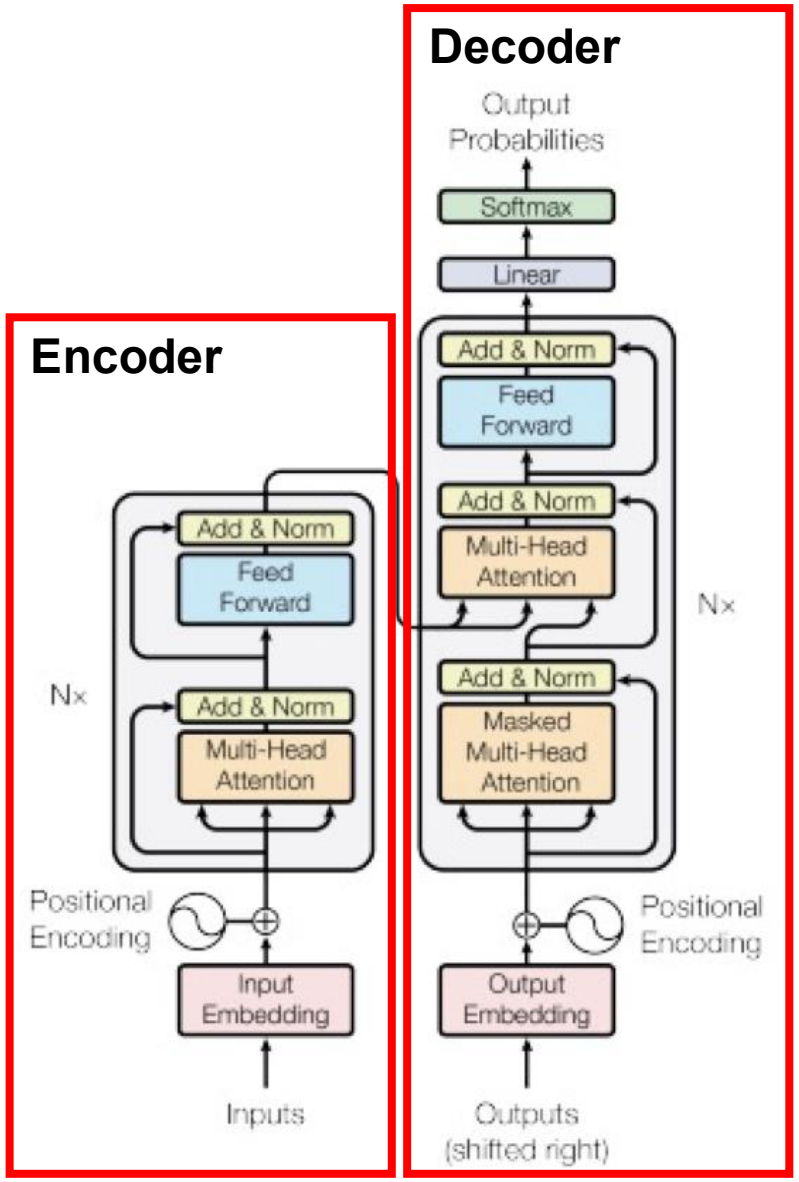


Figures from "Attention is All You Need" © Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin. All rights reserved. This content is excluded from our Creative Commons license. For more information, see https://ocw.mit.edu/help/faq-fair-use.

# Attendion Is All You Need

In this paper, the authors also tackle the task of translation with an encoder-decoder model.

- Through an attention mechanism the authors showed that you can construct powerful sequence models without recurrent layers.

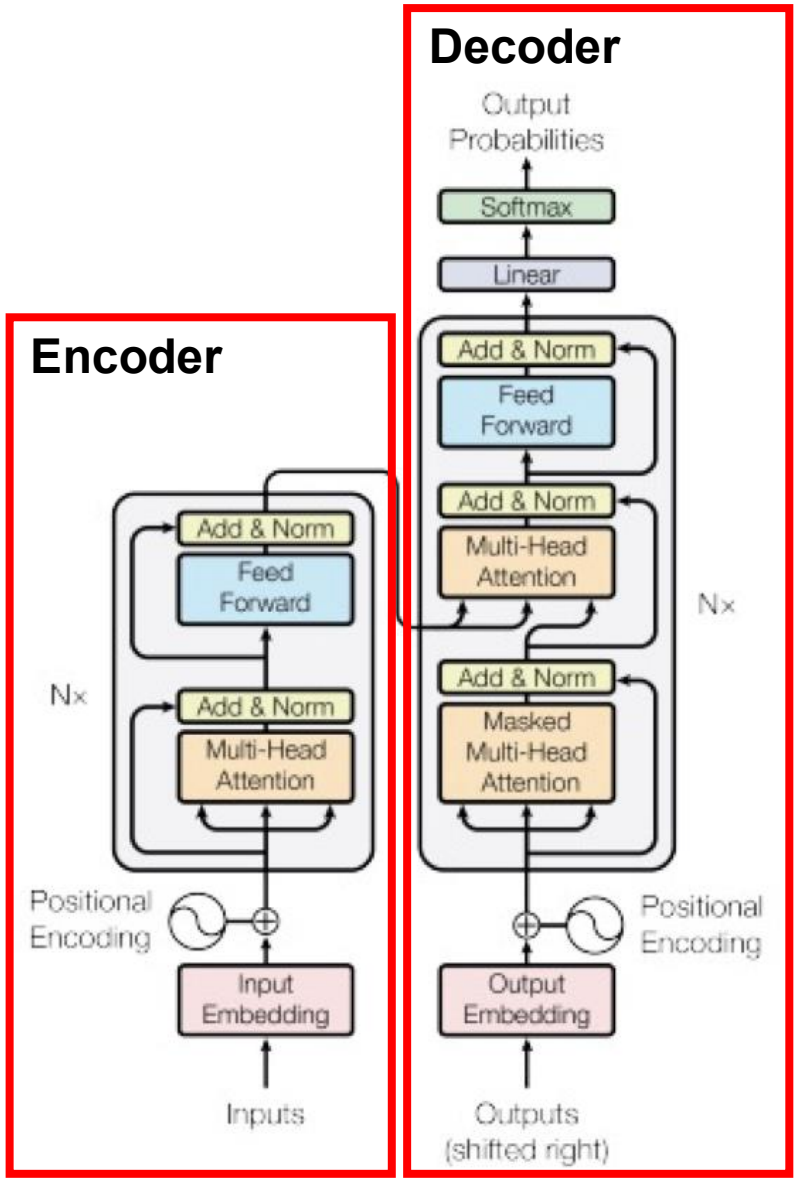


# Transformer Structure at a Glance

## Data Flow:

Input Tokens → Token + Positional Embedding

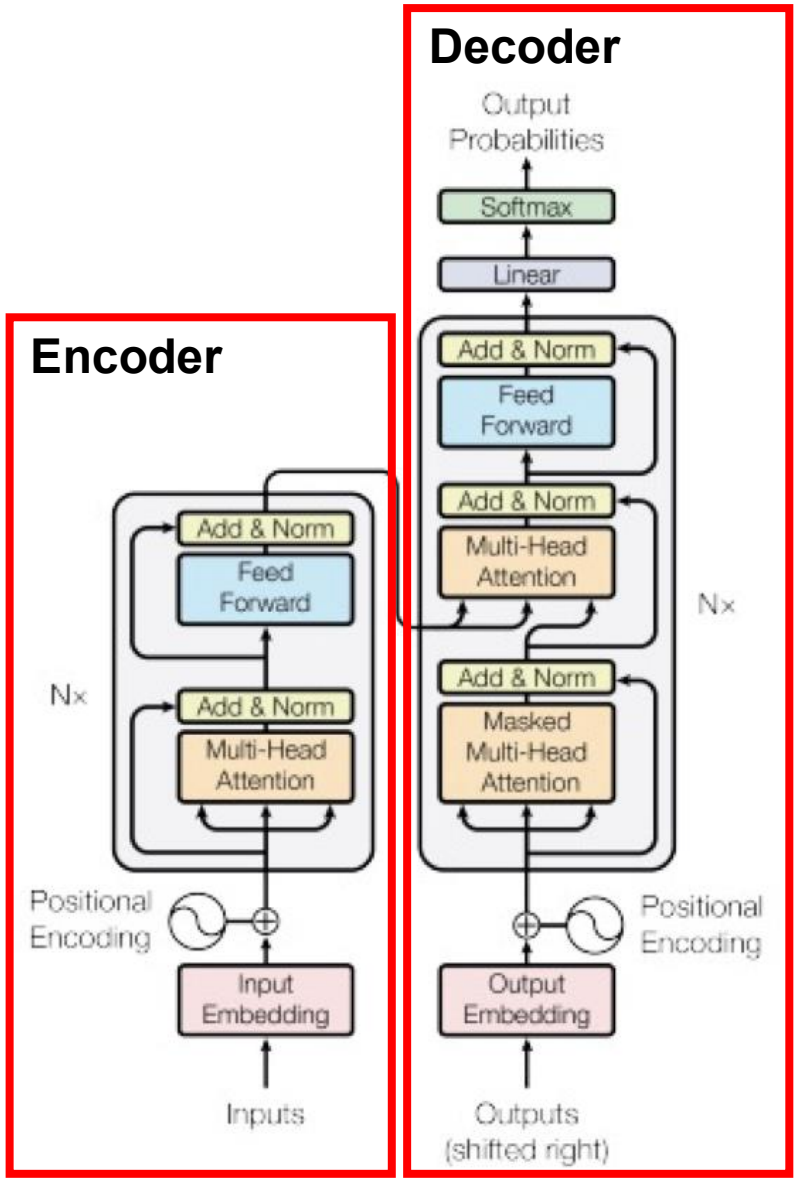
- Encoder stack builds contextual representations of the input.
- Decoder stack generates outputs one token at a time.
- Final linear layer + softmax maps hidden states to token probabilities.



# Transformer Structure at a Glance

Think of the transformer as two coordinated systems:

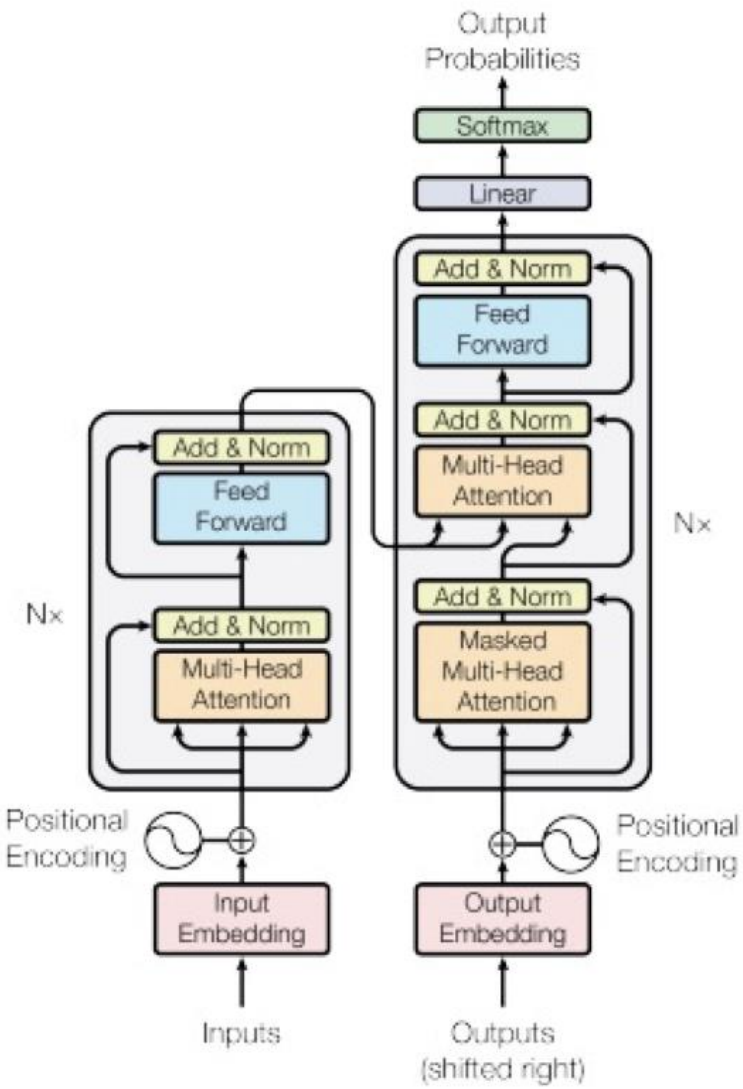
- **Understand the source** (encoder).
- **Generate the target** (decoder).



# Inside a Transformer Block

Breakdown of the layers:

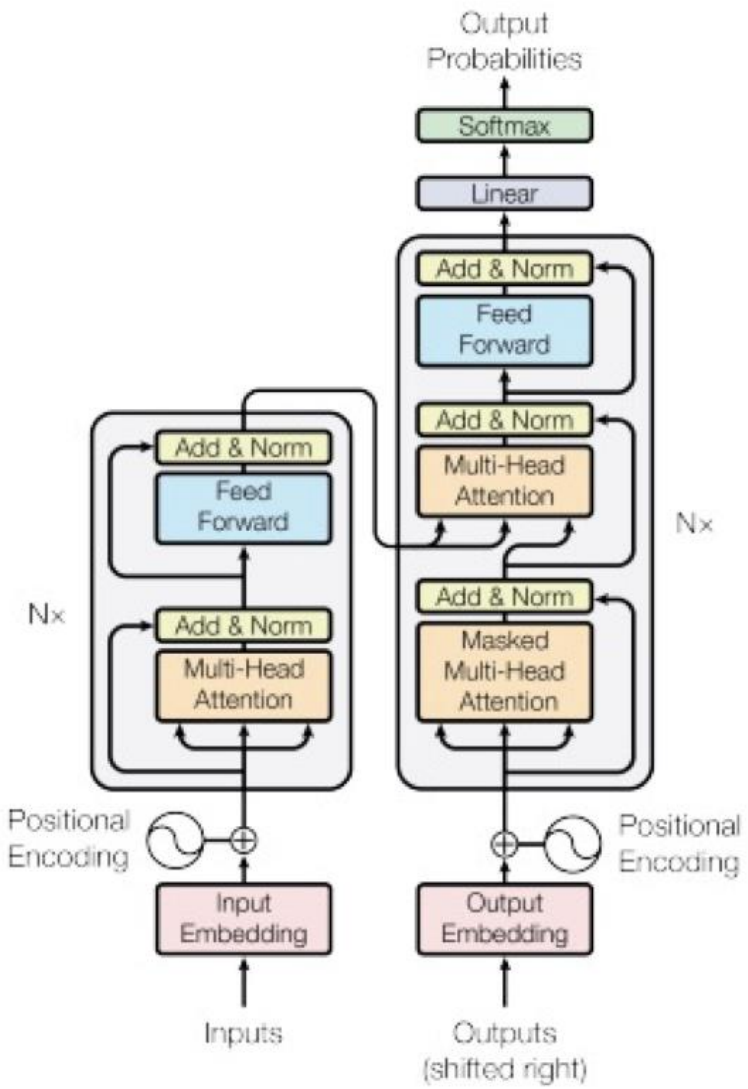
- Multi-head attention: lets each token gather relevant context from other tokens.
- Feed-forward network (FFN): transforms each token representation to add expressive power.
- Residual connections: preserve earlier information and stabilize deep stacks.
- Layer normalization: keeps activations in a stable range for training.



# Inside a Transformer Block

Why repeat blocks?

- Each layer refines representations, moving from surface patterns to richer structure.



# Why This Structure Works

The intuition behind each design choice is roughly as follows:

- Attention instead of recurrence: parallel processing + direct long-range connections.
- Multiple heads: different relationship types can be learned at the same time.
- Positional information: injects token order, since attention alone is order-agnostic.
- Encoder-decoder cross-attention: decoder can focus on the most relevant source tokens while generating.
- Masking in decoder self-attention: prevents peeking at future tokens.

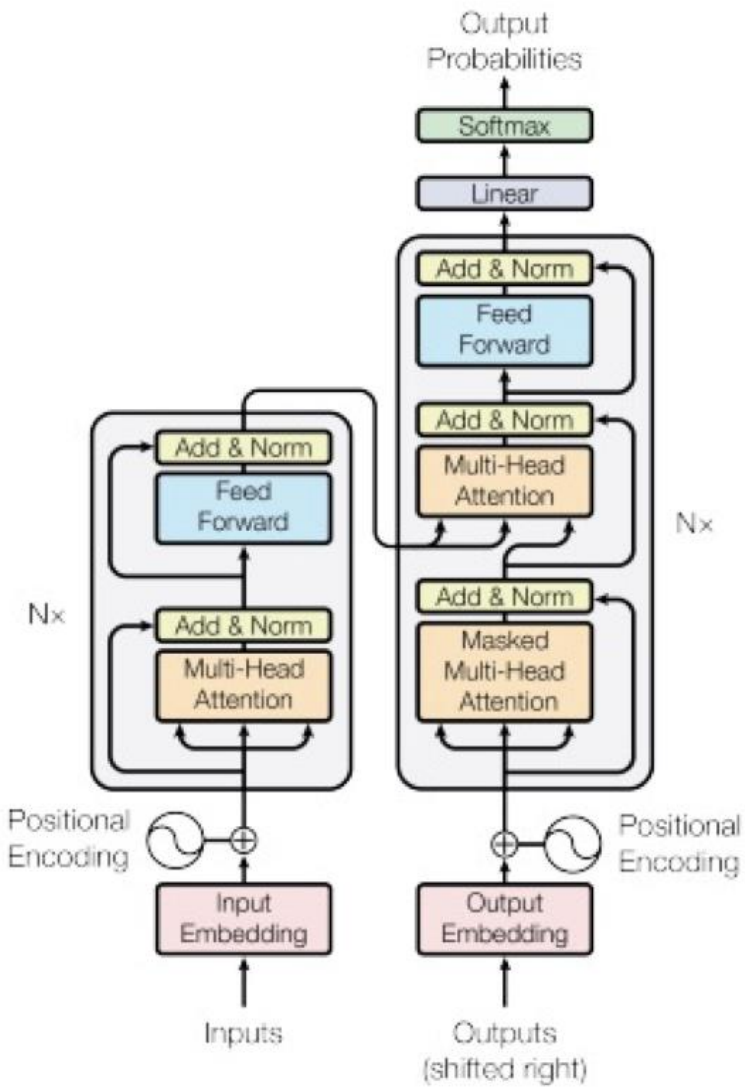
Mental model:

- Self-attention decides **what to look at**.
- FFN decides **how to transform what was gathered**.

# Why This Structure Works

Mental model:

- Self-attention decides what to look at.
- FFN decides how to transform what was gathered.



# Traditional Transformer Data Flow

**Stage 1: Source text -> encoder input**

- Tokenize source sentence into token ids.
- Convert ids to embeddings.
- Add positional information.
- Each token is now a vector representation encoding word meaning along with position.
- These vectors are the input to encoder layer 1.

Remember:

Self-attention does not start from raw words.

It starts from these learned token vectors.

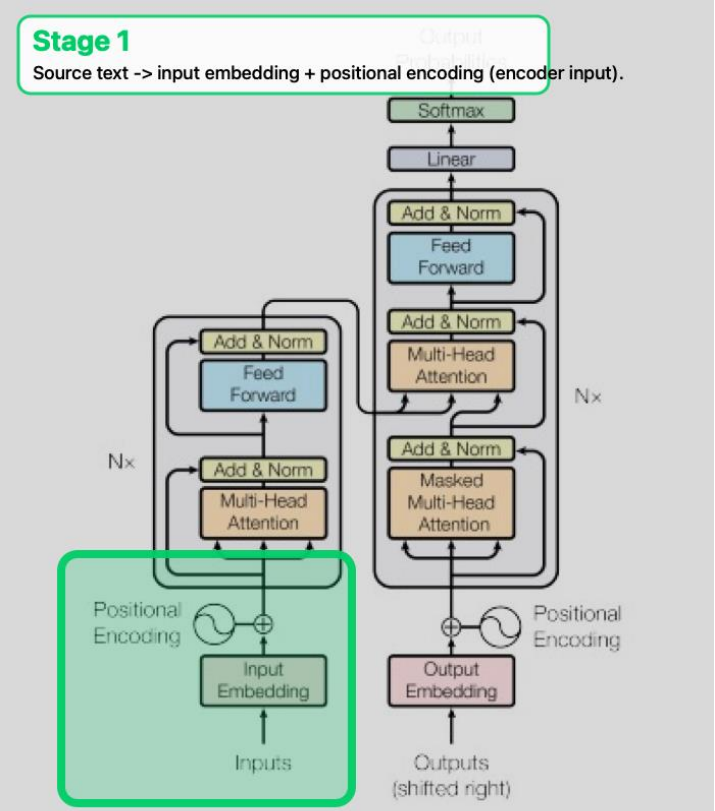



Figure 1: The Transformer - model architecture.

# Traditional Transformer Data Flow

**Stage 2: Encoder self-attention**

- In each encoder layer, create Query, Key and Value vectors from current source vectors.
- Each source token attends to all source tokens.
- Attention mixes useful context into each token representation.
- Residual connection + layer norm stabilize the update.

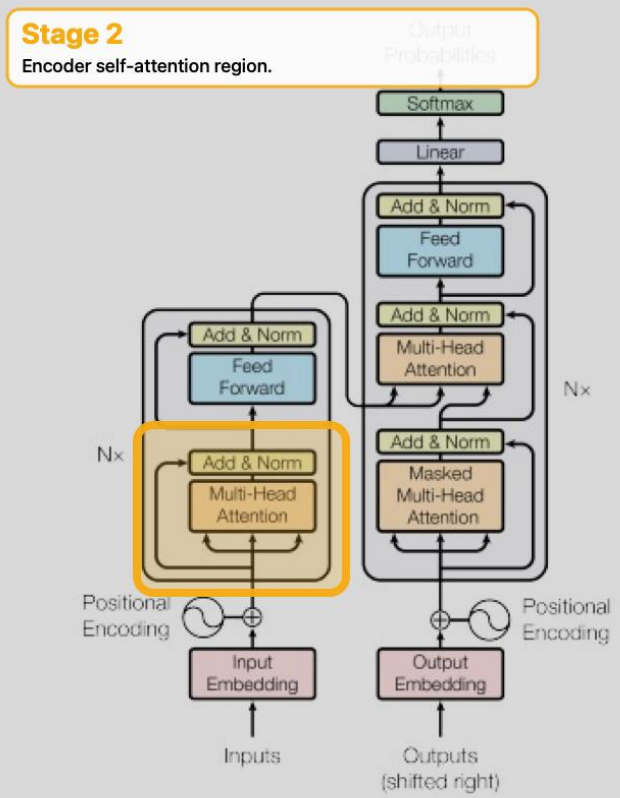



Figure 1: The Transformer - model architecture.

# Self-Attention Intuition

**Where Query, Key, Value come from**

- For each token vector `x`, the model applies three learned linear layers:
- `Q = xWq`, `K = xWk`, `V = xWv`.
- `Wq`, `Wk`, `Wv` are just trainable weight matrices.
- They are learned during training, like any other layer weights.

Operational view:

- Query: what this token wants to find.
- Key: what this token makes available to be matched.
- Value: the information this token can contribute.

# Self-Attention Intuition

**How tokens use Q, K, V to share information**

- Compare one token's Query with all tokens' Keys.
- High match score => pay more attention to that token.
- Use those scores to take a weighted mix of the Value vectors.
- Result: each token representation is updated using relevant context.

Mental model:

- Scores decide **who to listen to**.
- Values decide **what gets said**.

# Why multiple heads exist

A single head gives one attention map per token.

But language has multiple relationships at once.

In one sentence you may need syntax, entity links, and negation cues.

One map is often too limited to represent all of these clearly.

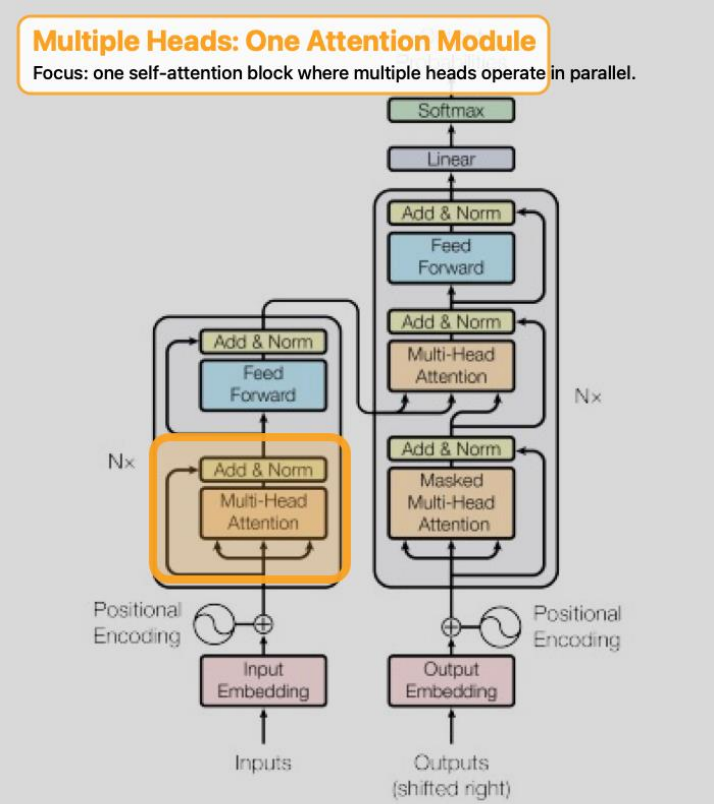



Figure 1: The Transformer - model architecture.

# Why multiple heads exist

A single attention map is often too limited for language.

Different heads can learn different relationship types in parallel.

Outputs from heads are combined, then projected.

Result: richer token representations.

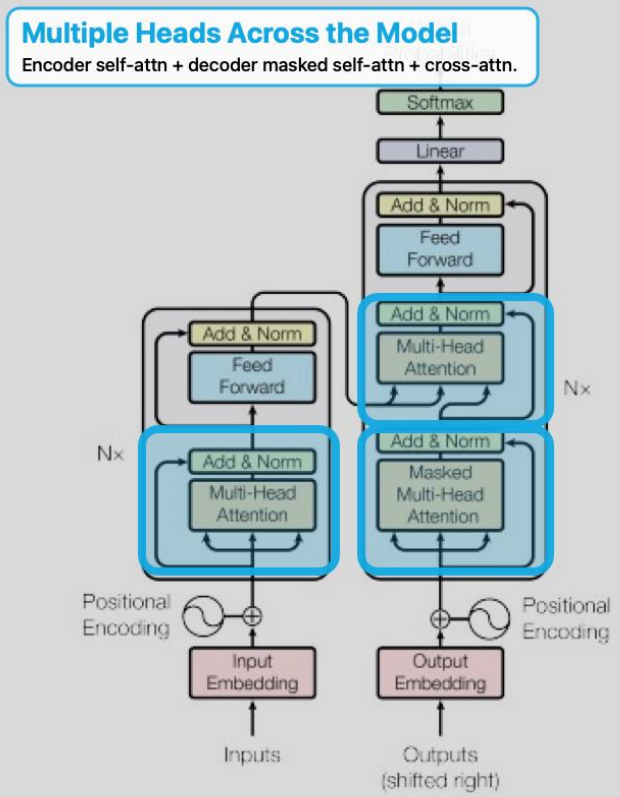



Figure 1: The Transformer - model architecture.

# Traditional Transformer Data Flow

**Stage 3: Encoder feed-forward + stacking**

Apply position-wise FFN to each token vector.

Use residual + layer norm again.

Repeat attention + FFN blocks for N encoder layers.

Final encoder output is often called the memory.

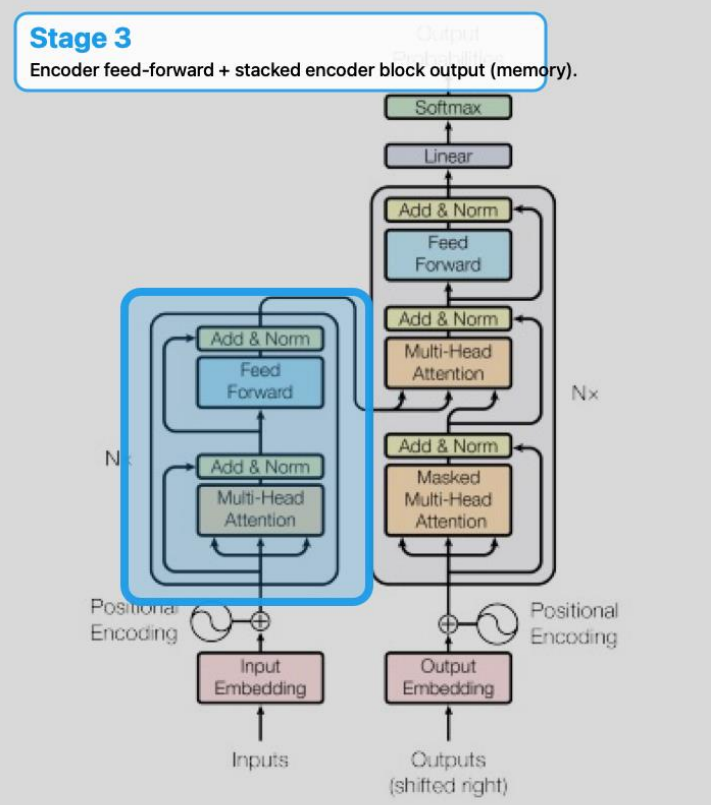



Figure 1: The Transformer - model architecture.

# Traditional Transformer Data Flow

**Stage 4: Decoder input and masked self-attention**

Decoder receives target tokens shifted right (during training).

Target embeddings + positional information are created.

Masked self-attention blocks access to future tokens.

This enforces realistic generation at train and inference time.

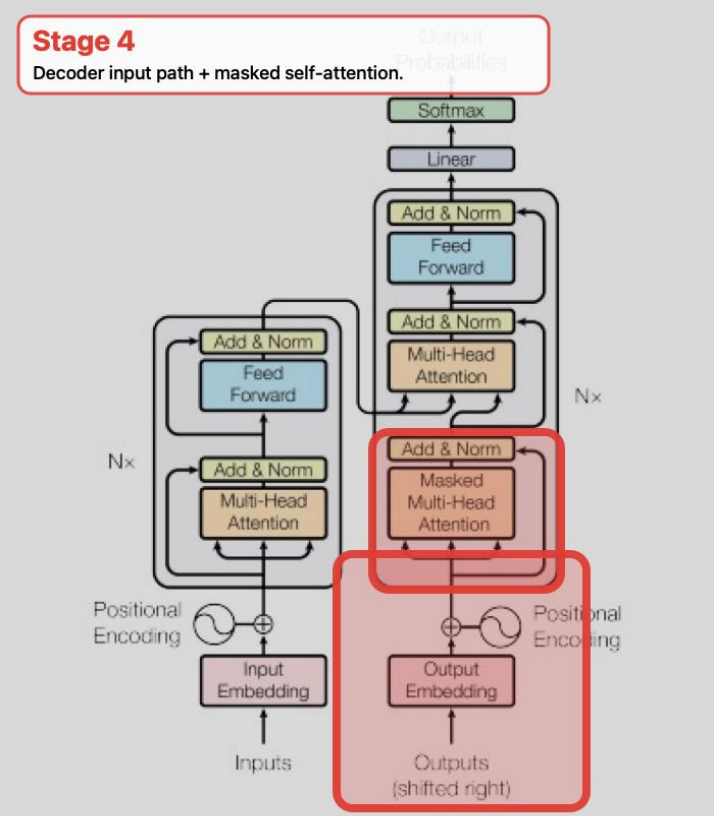



Figure 1: The Transformer - model architecture.

# Traditional Transformer Data Flow

**Stage 5: Cross-attention (decoder <-> encoder)**

Decoder creates Queries from its current hidden states.

Keys and Values come from encoder memory.

Decoder attends to relevant source tokens while generating next output.

Then FFN + residual + layer norm refine decoder states.

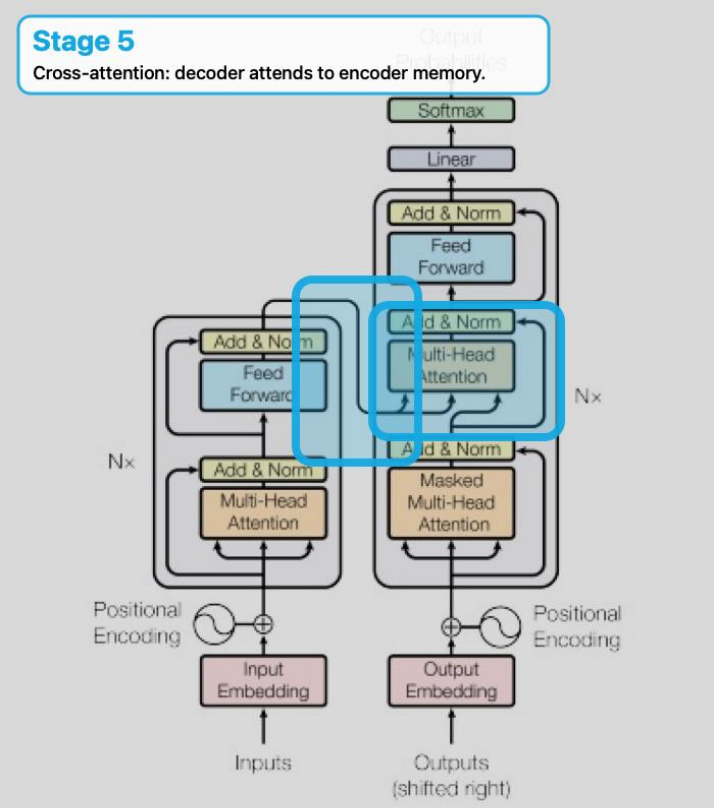



Figure 1: The Transformer - model architecture.

# Traditional Transformer Data Flow

**Stage 6: Output head and autoregressive loop**

Linear layer maps decoder states to vocabulary logits.

Softmax gives next-token probabilities.

Select next token (argmax/sampling) and append to decoder input.

Repeat until end token or max length.

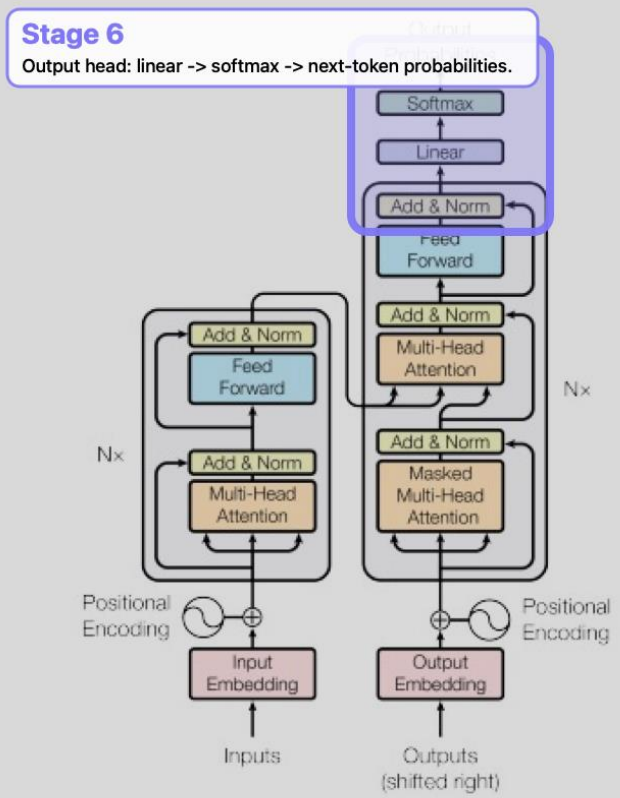



Figure 1: The Transformer - model architecture.

# Transformer Family Types

Language models which are built from the transformer architecture don’t always use both Encoder ad Decoder parts. In fact:

- Encoder (BERT-like): bidirectional understanding tasks.
- Decoder (GPT-like): autoregressive generation.
- Encoder-decoder (T5-like): text-to-text transformation.

# Transformer Family

## Encoder-Only
BERT-style

**Flow**
Input -> Encoder stack -> task head

**Use Case**
Best for understanding tasks

## Decoder-Only
GPT-style

**Flow**
Prompt -> Decoder stack -> next token

**Use Case**
Best for free-form generation

## Encoder-Decoder
T5/BART-style

**Flow**
Input -> Encoder; Output tokens -> Decoder

**Use Case**
Best for text-to-text transforms

# Encoder-Only Intuition (BERT-like)

Think of encoder-only models as deep readers.

- They see the whole input at once (bidirectional context).
- They build rich contextual representations for each token.
- They are strongest when output is understanding-focused.

Good fit:

- Classification, NER, extractive QA, retrieval features.

# Decoder-Only Intuition (GPT-like)

Think of decoder-only models as next-token writers.

- They generate left-to-right.
- At each position, they can only look at previous tokens.
- They are strongest for open-ended generation.

Good fit:

- Chat, creative writing, code generation, continuation.

# Encoder-Decoder Intuition (T5/BART-like)

Think of encoder-decoder models as read, then write.

- Encoder reads and compresses source information into memory.
- Decoder generates target text step-by-step.
- Cross-attention lets decoder fetch relevant source details each step.

Good fit:

- Translation, summarization, structured text transformation.

# Which Family Should I Choose?

| Task type | Best family | Why |
|---|---|---|
| Sentiment / topic / NER | Encoder-only | Strong bidirectional understanding |
| Free-form continuation / chat | Decoder-only | Natural left-to-right generation |
| Translation / summarization | Encoder-decoder | Explicit source->target mapping |

# Hugging Face Model Mapping

Models used in today's labs

- Encoder: `bert-base-uncased` for `fill-mask`.
- Decoder: `gpt2` for `text-generation`.
- Encoder-decoder: `t5-small` for `text2text-generation`.

# Safe Model Loading Policy

We shall stick to using `safetensors` when loading models and for reproducible loading the same revision of the model. This means we will:

- Verify `.safetensors` exists before loading.
- Load with `use_safetensors=True`.
- Set `trust_remote_code=False`. (default anyway)
- Pin `revision`.

# Summary

**Key message**

- Architecture understanding improves model selection and debugging.
- `pipeline()` is useful, but knowing internals improves reliability and trust.
- Week 6 will build on this with practical fine-tuning workflows.